EECE 2140

Computing Fundamentals for Engineers

System Design Overview

Elevator Scheduling System

C++ Simulation of Multi-Elevator Request Handling

Team Members: **Daniel Brightman**
Joel Kamhazi

Instructor: Dr. Fatema Nafa

Semester: Spring 2026

Repository: `github.com/dbrightman/EECE2140_final_project`

Contents

1 Project Overview **2**
1.1 Motivation . . . 2
1.2 Project Goals . . . 2

2 Basic Functionalities **2**
2.1 Functionality Summary Table . . . 2
2.2 Detailed Functionality Descriptions . . . 3

3 Object-Oriented Programming (OOP) Design **4**
3.1 Class Relationship Overview . . . 4
3.2 OOP Design Table . . . 5
3.3 Key OOP Requirements . . . 7

4 System Overview — Input, Process, Output **7**
4.1 High-Level Description . . . 7
4.2 IPO Diagram . . . 7
4.3 Detailed Step Descriptions . . . 7
4.3.1 Input . . . 8
4.3.2 Process . . . 8
4.3.3 Output . . . 8

5 Pseudo Code **8**

6 C++ Implementation Highlights **9**
6.1 Implementation Snapshot . . . 11

7 Challenges, Solutions & Lessons Learned **11**
7.1 Challenges & Solutions . . . 11
7.2 Lessons Learned . . . 12

8 Limitations & Future Work **12**

9 GitHub Repository Structure **12**

References **13**

A Sample Program Output **13**

B Selected Function Example **14**

1 Project Overview

Project Description

The **Elevator Scheduling System** is a terminal-based `C++` program that models how a building with multiple elevators responds to floor requests. The system accepts user input for building size, elevator starting positions, and requested floors; assigns each request to the closest elevator; computes a visit path using FCFS, SSTF, or SCAN. It then simulates elevator movement floor by floor. The project demonstrates object-oriented design using separate classes for elevator state, scheduling, building, and simulation output.

1.1 Motivation

Elevators are located in most buildings, and the logic behind their movement hasn't changed in decades. We wanted to create a simulation to test different movement algorithms for efficiency. A poor scheduling strategy can increase travel distance, waste time, and make the user's experience worse. Because real elevator systems are expensive and difficult to experiment with directly, simulation is a practical way to test scheduling strategies, observe behavior, and reason about system design in a controlled environment.

1.2 Project Goals

- Build a working `C++` program that models a building, multiple elevators, and a queue of floor requests.
- Apply object-oriented programming by separating the system into classes with clear responsibilities.
- Implement three scheduling algorithms: FCFS, SSTF, and SCAN.
- Simulate elevator movement one floor at a time and display each step in the terminal.
- Validate user input and produce readable terminal output before and after scheduling.
- Document the design clearly in a final report and presentation.

2 Basic Functionalities

This section summarizes the core functions of the Elevator Scheduling System and explains how each one contributes to the overall program.

2.1 Functionality Summary Table

#	Functionality	Description	Input	Output
F1	**Floor Request Input & Validation**	Accepts a requested floor, checks that it is within the building's allowed range, and either stores the request or returns an error.	Elevator ID or requested floor	Updated queue or error message
F2	**Best Elevator Assignment**	Compares each elevator's current floor to the requested floor and assigns the request to the closest one.	Requested floor, elevator positions	Assigned elevator ID
F3	**Path Scheduling**	Computes the order of floors to visit using FCFS, SSTF, or SCAN based on the user's selected algorithm.	Request queue, current floor, direction	Ordered visit path
F4	**Simulation & Status Reporting**	Moves the elevator through the path one floor at a time, prints each step, stores a movement log, and shows final position.	Elevator path, starting floor	Step-by-step output, final status summary

2.2 Detailed Functionality Descriptions

⌨ F1 — Floor Request Input & Validation

Description: The program accepts floor requests from the user and checks that each request falls within the valid range of the building. Invalid values immediately stop the program so they do not corrupt the request queues or create impossible elevator movements.

Input: Elevator ID and floor number, or a building-wide floor request.
Output: Updated request queue or a descriptive error message.
Contribution: Ensures the rest of the program operates on valid inputs.

↕ F2 — Best Elevator Assignment

Description: The `Building` class assigns requests to elevators by finding the closest ones. It measures the absolute distance between each elevator's current floor and the requested floor, then selects the elevator with the smallest distance.

Input: Requested floor and the current floors of all elevators.
Output: ID of the assigned elevator.

Contribution: Provides a simple but effective dispatch strategy for multiple elevators.

F3 — Path Scheduling

Description: The `Scheduler` class determines the order in which an elevator should visit requested floors. FCFS preserves request order, SSTF repeatedly chooses the nearest next floor, and SCAN goes all the way up or down before reversing direction.

Input: Request queue, current floor, and direction for SCAN.
Output: Ordered vector of floors to visit.
Contribution: Lets the project compare different scheduling strategies within one common program structure.

F4 — Simulation & Status Reporting

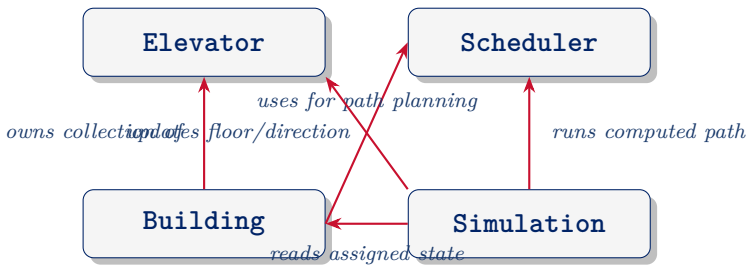
Description: After a path has been calculated, the `Simulation` class moves the elevator one floor at a time until it reaches every requested destination. Each step is printed and stored in a log, and the elevator's final floor and direction are updated when the run is complete.

Input: Elevator object and computed path.
Output: Floor-by-floor movement trace, total step count, simulation log, and final status.
Contribution: Turns the scheduling logic into visible behavior that can be tested and discussed.

3 Object-Oriented Programming (OOP) Design

3.1 Class Relationship Overview

The system is built around four primary classes: `Elevator`, `Scheduler`, `Building`, and `Simulation`. The design follows a straightforward separation of responsibilities. `Elevator` stores the state of one elevator, `Scheduler` handles path computation, `Building` manages the collection of elevators and request assignment, and `Simulation` executes movement and logging.



3.2 OOP Design Table

Class	Attribute / Method	Access	Data Type	Description
Elevator	`elevatorID`	private	`int`	Unique identifier for each elevator
	`currentFloor`	private	`int`	Current floor position
	`direction`	private	`string`	Current direction: up, down, or idle
	`requestQueue`	private	`vector<int>`	List of pending floor requests
	`addRequest(int)`	public	`void`	Adds a floor request to the queue
	`clearRequests()`	public	`void`	Removes all finished requests
	`setCurrentFloor(int)`	public	`void`	Updates elevator position
	`setDirection(string`	public	`void`	Updates movement direction
	`getCurrentFloor()`	public	`int`	Returns current floor
	`displayStatus()`	public	`void`	Prints elevator status to terminal
Scheduler	`algorithmType`	private	`string`	Stores selected scheduling algorithm
	`setAlgorithm(string`	public	`void`	Changes the scheduling algorithm
	`getAlgorithm()`	public	`string`	Returns selected algorithm name
	`computeFCFS()`	public	`vector<int>`	Returns floors in request order
	`computeSSTF()`	public	`vector<int>`	Returns nearest-floor-first order
	`computeSCAN()`	public	`vector<int>`	Returns sweep-based order
	`getOptimalPath()`	public	`vector<int>`	Calls the correct scheduling method

Class	Attribute / Method	Access	Data Type	Description
	`Scheduler(string)`	public	constructor	Initializes the scheduler
	`totalFloors`	private	`int`	Total number of floors in the building
	`elevators`	private	`vector<Elevat`	Collection of elevator objects
`Building`	`addElevator(int)`	public	`void`	Adds a new elevator to the building
	`addRequestToElevato`	public	`void`	Sends a request to one chosen elevator
	`assignBestElevator()`	public	`int`	Finds the closest elevator to a request
	`requestFloor(int)`	public	`void`	Validates and dispatches a request
	`getElevatorByID()`	public	`Elevator*`	Returns pointer to matching elevator
	`displayAllStatus()`	public	`void`	Prints the status of all elevators
	`getTotalFloors()`	public	`int`	Returns building floor count
	`stepCount`	private	`int`	Counts total movement steps
	`log`	private	`vector<string`	Stores movement events as text
	`Simulation()`	public	constructor	Initializes step counter and log
	`run()`	public	`void`	Simulates floor-by-floor movement
	`printLog()`	public	`void`	Prints all saved events
	`reset()`	public	`void`	Clears old simulation data

3.3 Key OOP Requirements

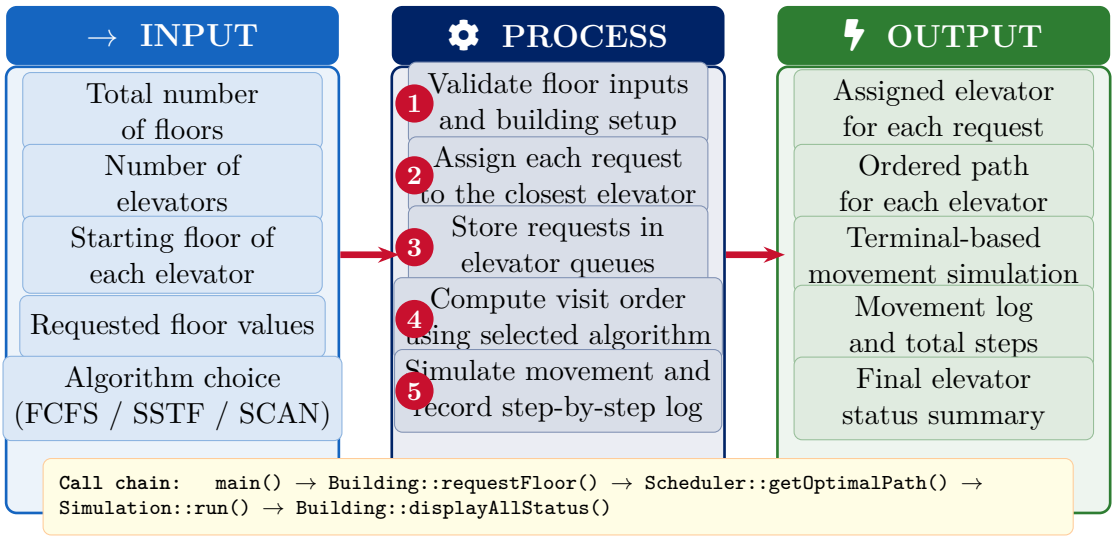
- **At least four classes:** The project includes `Elevator`, `Scheduler`, `Building`, and `Simulation`.
- **Private attributes:** Core data such as elevator position, direction, request queues, and step logs are kept private.
- **Encapsulation:** Each class exposes only the operations needed by the rest of the program.
- **Separation of responsibilities:** Scheduling logic is isolated from building management and simulation output.
- **STL usage:** The implementation uses `vector<int>` for request queues, `vector<Elevator>` for storing elevators, and `vector<string>` for the event log.

4 System Overview — Input, Process, Output

4.1 High-Level Description

The program follows an Input → Process → Output structure. The user enters building information, elevator starting positions, requested floors, and a scheduling algorithm. The system validates that input, assigns requests to elevators, computes visit paths, and then simulates the resulting movement. Finally, it prints the event log and final status summary.

4.2 IPO Diagram



4.3 Detailed Step Descriptions

4.3.1 Input

→ Total number of floors in the building.

→ Number of elevators and starting floor for each elevator.

→ Number of requested floors and the floor values themselves.

→ Scheduling algorithm choice: FCFS, SSTF, or SCAN.

4.3.2 Process

Step 1. Validate the building setup and requested floors.

Step 2. Assign each request to the elevator whose current floor is closest.

Step 3. Store requests in the selected elevator's queue.

Step 4. Compute each elevator's path using the chosen scheduling algorithm.

Step 5. Simulate elevator movement one floor at a time and record every step.

4.3.3 Output

→ Assigned elevator IDs for incoming requests.

→ Ordered floor paths for each elevator.

→ Step-by-step terminal simulation of movement.

→ A saved simulation log and a final status summary showing ending floors and empty queues.

5 Pseudo Code

Main Simulation Loop

```
PROGRAM ElevatorSystem
  INPUT floors, elevators, requests, algorithm
  CREATE building and elevators

  FOR each request:
    ASSIGN request to best elevator
  END FOR

  FOR each elevator:
    GET queue
    COMPUTE path using selected algorithm
    RUN floor-by-floor simulation
    CLEAR queue
  END FOR

  PRINT movement log
  PRINT final elevator status
END PROGRAM
```

Sub-Function: assignBestElevator(requestedFloor)

```
FUNCTION assignBestElevator(requestedFloor)
  bestID = -1
  minDistance = infinity

  FOR each elevator:
    distance = abs(currentFloor - requestedFloor)
    IF distance < minDistance:
      minDistance = distance
      bestID = elevatorID
    END IF
  END FOR

  RETURN bestID
END FUNCTION
```

Sub-Function: getOptimalPath(queue)

```
FUNCTION getOptimalPath(queue)
  IF FCFS:
    RETURN queue
  ELSE IF SSTF:
    RETURN nearest-floor-first order
  ELSE IF SCAN:
    RETURN sweep order
  END IF
END FUNCTION
```

6 C++ Implementation Highlights

Simulation::run() - Core C++ Implementation

```
void Simulation::run(Elevator& elevator, const std::vector<int>& path) {
    if (path.empty()) {
        std::cout << "No path to simulate.\n";
        return;
    }

    int position = elevator.getCurrentFloor();

    for (size_t i = 0; i < path.size(); i++) {
        int target = path[i];

        while (position != target) {
            std::string direction;
            if (target > position) {
                position++;
                direction = "up";
            } else {
```

```
                position--;
                direction = "down";
            }

            stepCount++;
            elevator.setDirection(direction);

            std::string event = "Step " + std::to_string(stepCount) +
                                ": Elevator " + std::to_string(elevator.
    getElevatorID()) +
                                " at floor " + std::to_string(position) +
                                " moving " + direction;

            std::cout << event << std::endl;
            log.push_back(event);
        }

        std::cout << ">>> Arrived at requested floor: " << target << std::endl;
    }

    elevator.setCurrentFloor(position);
    elevator.setDirection("idle");
}
```

Key Design Decisions

D1. Separate classes for separate jobs: `Building` handles dispatch, `Scheduler` handles path ordering, `Elevator` stores elevator state, and `Simulation` handles movement and logging.

D2. Direct object storage with vectors: The project stores elevators directly in `vector<Elevator>` and floor requests in `vector<int>`, which keeps the design simple and readable.

D3. Algorithm selection through one interface: `Scheduler::getOptimalPath()` provides a single entry point for FCFS, SSTF, and SCAN.

D4. Readable debugging output: The helper `printPath()` and the saved movement log make it much easier to verify correctness from the terminal.

6.1 Implementation Snapshot

Metric	Project Value
Classes fully implemented	4 core classes: Elevator, Scheduler, Building, Simulation
Scheduling algorithms supported	3: FCFS, SSTF, SCAN
Functions / methods in project	32 total, including class methods plus `main()` and `printPath()`
Memory management approach	No manual `new`/`delete`; objects stored directly in STL containers
Terminal output style	Queue display, calculated paths, movement log, and final status summary

7 Challenges, Solutions & Lessons Learned

7.1 Challenges & Solutions

#	Challenge	Solution
1	Organizing multiple responsibilities such as building setup, elevator state, scheduling, and simulation output.	Divided the program into four classes so each part of the system had one clear purpose.
2	Supporting FCFS, SSTF, and SCAN while keeping the main program structure simple.	Created a dedicated `Scheduler` class with a single `getOptimalPath()` interface.
3	Handling invalid user input such as out-of-range floors or invalid building settings.	Added validation checks in `main()` and `Building::requestFloor()`, and defaulted invalid starting floors to floor 1.
4	Making terminal output easy to read during debugging and demo.	Added `printPath()` for queue display and a movement log inside `Simulation`.

7.2 Lessons Learned

- **OOP makes medium-sized programs much easier to manage.** Once the classes were separated properly, debugging and extending the system became much more straightforward.
- **Scheduling algorithms genuinely affect behavior.** Even in a small simulation, FCFS, SSTF, and SCAN can produce noticeably different paths.
- **Input validation matters.** A simple simulation can still fail in confusing ways if invalid values are accepted without checks.
- **Readable output improves debugging.** Printing queues, paths, and movement steps made it much easier to verify that the logic matched the expected result.
- **Simulation is a useful engineering tool.** The project showed how code can represent a real system and help compare different decision strategies without physical testing.

8 Limitations & Future Work

Current Limitation	Proposed Future Extension
Console-only output.	Add a graphical interface that shows elevators moving between floors in real time.
No elevator capacity limit.	Add passenger count, weight limit, and more realistic loading behavior.
Basic scheduling algorithms only.	Extend the scheduler with priority-based, dynamic, or AI-assisted strategies.
Simple request handling.	Add more advanced multi-elevator coordination so requests can be reassigned dynamically.
No passenger modeling.	Introduce passenger objects with individual wait times, origins, and destinations.

9 GitHub Repository Structure

Repository Layout

```
EECE2140_final_project/
|-- images
|-- src
    |-- Building.h
    |-- Building.cpp
    |-- Elevator.h
```

```
    |-- Elevator cpp
    |-- Scheduler h
    |-- Scheduler cpp
    |-- Simulation h
    |-- Simulation cpp
    |-- main cpp
|-- README md
`-- docs/
    `-- SystemDesignOverview pdf
```

References

[1] Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.

[2] Meyers, S. (2014). *Effective Modern C++*. O'Reilly Media.

[3] Cppreference.com. *std::vector*. https://en.cppreference.com/w/cpp/container/vector

[4] Cppreference.com. *std::sort*. https://en.cppreference.com/w/cpp/algorithm/sort

[5] Cppreference.com. *std::abs*. https://en.cppreference.com/w/cpp/numeric/math/abs

A Sample Program Output

Expected Console Output - Example Run

```
Enter total number of floors: 10
Enter number of elevators: 2
Enter starting floor for Elevator 1: 1
Enter starting floor for Elevator 2: 7
Enter number of floor requests: 3
Enter requested floor 1: 3
Elevator 1 assigned to floor 3.
Floor 3 added to Elevator 1 queue.
Enter requested floor 2: 9
Elevator 2 assigned to floor 9.
Floor 9 added to Elevator 2 queue.
Enter requested floor 3: 2
Elevator 1 assigned to floor 2.
Floor 2 added to Elevator 1 queue.

--- Elevator Status ---
Elevator 1 | Floor: 1 | Direction: idle | Queue: 3, 2
Elevator 2 | Floor: 7 | Direction: idle | Queue: 9
```

```
Choose scheduling algorithm (FCFS, SSTF, or SCAN): SCAN

========================================
Processing Elevator 1
Original queue: 3 -> 2
Calculated path using SCAN: 2 -> 3
Step 1: Elevator 1 at floor 2 moving up
>>> Arrived at requested floor: 2
Step 2: Elevator 1 at floor 3 moving up
>>> Arrived at requested floor: 3
Simulation complete. Total steps: 2

========================================
Processing Elevator 2
Original queue: 9
Calculated path using SCAN: 9
Step 3: Elevator 2 at floor 8 moving up
Step 4: Elevator 2 at floor 9 moving up
>>> Arrived at requested floor: 9
Simulation complete. Total steps: 4

--- Simulation Log ---
Step 1: Elevator 1 at floor 2 moving up
Step 2: Elevator 1 at floor 3 moving up
Step 3: Elevator 2 at floor 8 moving up
Step 4: Elevator 2 at floor 9 moving up

--- Elevator Status ---
Elevator 1 | Floor: 3 | Direction: idle | Queue: empty
Elevator 2 | Floor: 9 | Direction: idle | Queue: empty
```

B Selected Function Example

Building::assignBestElevator() - Short Dispatch Function

```cpp
int Building::assignBestElevator(int requestedFloor) const {
    if (elevators.empty()) {
        std::cout << "Error: no elevators available.\n";
        return -1;
    }

    int bestID = -1;
    int minDistance = std::numeric_limits<int>::max();

    for (size_t i = 0; i < elevators.size(); i++) {
        int distance = std::abs(elevators[i].getCurrentFloor() - requestedFloor);

        if (distance < minDistance) {
            minDistance = distance;
            bestID = elevators[i].getElevatorID();
        }
```

```
    }

    return bestID;
}
```